

Task Management - Assignment 1

Fundamental Programming Techniques, Computer Science Department, TUCN

Spring 2026

1 Requirements

Goal: Design and implement an application for managing the tasks assigned to employees, starting from the class diagram below.

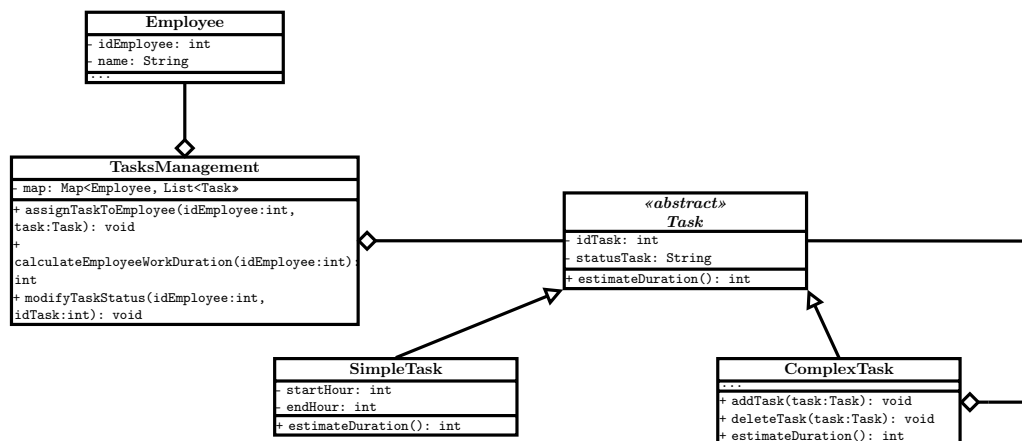
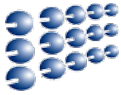


Figure 1: Task management UML class diagram

Implement the following classes:

1. **Employee**
2. **Task**: a sealed abstract class with an abstract method, `estimateDuration()`
3. **SimpleTask**: overrides the `estimateDuration()` method to return the estimated duration of a task based on the start and end hours
4. **ComplexTask**: can be composed of a simple and/or tasks
5. **TasksManagement**: with methods:
 - (a) `assignTaskToEmployee()`, which assigns a tasks to an employee
 - (b) `calculateEmployeeWorkDuration()`, which estimates the work duration of an employee based on the completed tasks (i.e., `statusTask = "Completed"`) assigned to them
 - (c) `modifyTaskStatus()`, which modifies the status of a task assigned to an employee to "Completed" or "Uncompleted" depending on the case
6. **Utility**: with the following methods:
 - (a) a method that filters all employees who have a work duration grater than 40 hours, sorts them in ascending order according to the work duration, and displays their names
 - (b) a method that computes, for each employee, the number of completed and uncompleted tasks such that it returns a map where the key is the employee's name and the value is a `Map<String, Integer>`, where the key represents the possible status of the tasks (i.e., "Completed" or "Uncompleted"), and the value is the number of tasks in that category.



A project manager of the software company can interact with the application through a graphical user interface and can perform the following operations:

1. Add employees, add task (simple or complex), assign task to employee
2. View employees and their tasks (including the estimation of duration)
3. Modify the status of task assigned to an employee
4. View the statistics provided by the methods of the `Utility` class.

All data will be persisted using the serialization technique.

Note: Implement constructors, getters/setters, and any additional classes, attributes, and methods needed to fully implement the required functionality.

2 Deliverables

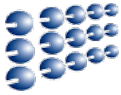
1. **Source code files:** will be uploaded on the personal **Gitlab** account created according to the instructions in the **Laboratory Resources** document, and following steps:
 - (a) Create a private repository on **Gitlab** named according to the following template
PT2026_Group_LastName_FirstName_Assignment_1
 - (b) Push the source code (**not as archive**).
 - (c) Share the repository with the user **utcn_dsrl** with Maintainer as role.
2. The **draw.io** file with the UML use case diagram, package diagram and the class diagram will be uploaded in the repository with the source code files.

3 Evaluation

The assignment will be graded as follows:

Requirements	Grading
1. Use an object-oriented programming design 2. Use the layered architectural pattern 3. Implement the classes: <code>Employee</code>, <code>Task</code>, <code>SimpleTask</code>, <code>ComplexTask</code> 4. Implement a graphical user interface using Java Swing, such that the project manager can add and view employees and tasks 5. Implement serialization for data persistence 6. Use <code>foreach</code> instead of <code>for(int i = 0; ...)</code> 7. Implement classes with maximum 300 lines (except UI classes) and methods with maximum 30 lines 8. Use the Java naming conventions 9. Correct UML diagrams	5 points
Implement the method <code>assignTaskToEmployee()</code> and the corresponding functionality in the GUI	+1 point
Implement the method <code>calculateEmployeeWorkDuration()</code> and the corresponding functionality in the GUI	+1 point
Implement the method <code>modifyTaskStatus()</code> and the corresponding functionality in the GUI	+1 point
Implement the filtering method from the <code>Utility</code> class point (a) and the corresponding functionality in the GUI	+1 point

Continued on next page



Requirements (continued)	Grading
Implement the method from <code>Utility</code> class point (b) that computes, for each employee, the number of completed and uncompleted tasks and the corresponding functionality in the GUI	+1 point

4 Bibliography

1. **Swing:** <https://docs.oracle.com/javase/tutorial/uiswing/index.html>
2. **Java serialization:** <https://www.baeldung.com/java-serialization>
3. **Composite design pattern:** <https://www.baeldung.com/java-composite-pattern>
4. **Java sealed classes:** <https://www.baeldung.com/java-sealed-classes-interfaces>